

RESUMO PARA ESTUDO - AVALIAÇÃO OBJETIVA 1

Este resumo é apenas um norteador do conteúdo que estará na Avaliação Objetiva 1. Utilize o texto das aulas para aprofundar o conhecimento.

1. Concepção do Produto e Mentalidade Startup

1.1 O Cenário de Inovação e a "Dor" do Usuário

No desenvolvimento moderno, adotamos o conceito de **Situação de Aprendizagem (SA)** como uma incubadora de startups. O software não deve ser visto como um fim, mas como um meio para resolver problemas reais.

- **Foco no Usuário:** Assim como o TikTok resolveu a "dor" da busca por conteúdo entregando algoritmos proativos, o desenvolvedor deve identificar a necessidade real antes de abrir o editor.
- **Axioma do Arquiteto:** "Codificar o sistema errado é o maior bug que uma startup pode cometer." Construir algo tecnicamente perfeito que ninguém precisa é o erro mais caro da engenharia.

1.2 Canvas de Proposta de Valor

A modelagem do negócio deve preceder a codificação. Utilizamos o Canvas para alinhar a solução às necessidades do público:

- **Dores:** Frustrações e obstáculos que o software deve eliminar (ex: no Spotify, a dor era a pirataria complexa).
- **Ganhos:** Benefícios esperados (ex: música com um clique).
- **Tarefas do Cliente:** Atividades rotineiras do usuário.
- **Analgésicos e Criadores de Ganhos:** O software deve atuar nessas duas frentes para garantir valor de mercado.

2. Engenharia de Requisitos: A Ponte entre Ideia e Construção

2.1 Definição e o Dilema da Qualidade

A Engenharia de Requisitos conecta o desejo do cliente à execução técnica. O **Dilema da Qualidade** surge quando o sucesso do software é ameaçado por prazos irreais ou incompreensão das necessidades, resultando em insatisfação do usuário.

2.2 Requisitos Funcionais (RF) vs. Não Funcionais (RNF)

Categoria	Definição	Exemplos (Sistema CasaSegura)
Funcionais (RF)	O que o sistema deve fazer (funções de negócio).	Armar/desarmar alarme; Exibir planta da casa; Monitorar sensores.
Não Funcionais (RNF)	Atributos de qualidade, desempenho e restrições.	Reconhecer eventos em < 1s (Desempenho); Proteção via PDO (Segurança); Disponibilidade de 99% (Confiabilidade); Interface intuitiva (Usabilidade).

2.3 User Stories (Histórias de Usuário)

Nesta técnica, descrevemos funcionalidades sob a ótica do usuário: **"Como [Ator], eu quero [Funcionalidade] para que [Valor]"**.

- **Ator:** O papel de quem interage (Proprietário, Admin, etc.).

- **CrITÉrios de Aceitação:** Conjunto de condições que validam se a história foi concluída corretamente.
-

3. Modelagem UML e Design de Experiência (UX/UI)

3.1 Linguagem de Modelagem Unificada (UML)

Utilizamos o **Draw.io** ou ferramentas similares para documentar a arquitetura visual:

- **Diagrama de Casos de Uso:** Foca nos atores e funções. O relacionamento `<<include>>` indica dependência obrigatória de uma função por outra.
- **Diagrama de Classes:** Visão estrutural estática. Define classes (Nome, Atributos, Métodos) e relacionamentos: **Associação** (referência), **Agregação/Composição** (todo/parte) e **Generalização** (herança).

3.2 Fundamentos de Interface e UX

Aplicamos as "Regras de Ouro":

1. **Usuário no comando:** Evitar ações inesperadas.
2. **Redução da carga de memória:** Interfaces intuitivas que não exigem memorização excessiva.
3. **Consistência:** Elementos visuais padronizados em todas as telas.

3.3 Prototipação no Figma

Criamos wireframes de baixa fidelidade para validar o fluxo. Telas obrigatórias: **Login**, **Dashboard** (indicadores), **Listagem** (tabelas) e **Formulário CRUD** (inputs). Use o **WebAIM Contrast Checker** para validar a acessibilidade das cores desde o protótipo.

4. Arquitetura e Setup de Desenvolvimento

4.1 Stack Tecnológica: Justificativa Arquitetural

A escolha da stack baseia-se em maturidade e custo-benefício:

- **PHP:** Domina cerca de **77% da web**. É escolhido pela alta disponibilidade de mão de obra e infraestrutura de baixo custo.
- **MySQL:** Padrão de integridade de dados. A integração deve ser feita via **PDO (PHP Data Objects)** para prevenir ataques de **SQL Injection**.
- **JavaScript:** Essencial para interatividade client-side.

4.2 Ambiente Local (XAMPP e VS Code)

- **XAMPP:** Emula o servidor local (Apache, MariaDB, PHP).
 - **Dica de Especialista:** Caso o Apache não inicie, verifique o **Conflito na Porta 80** (comumente usada pelo Skype ou Docker). Altere para 8080 se necessário.
- **VS Code:** Utilize extensões como **PHP Intelephense** para diagnóstico em tempo real.

4.3 Versionamento Profissional

O Git é a ferramenta; o GitHub é a plataforma.

- **Estrutura de Pastas:** `/frontend`, `/backend`, `/docs`.
 - **.gitignore:** Fundamental para não subir arquivos desnecessários como `node_modules`, configurações locais de IDE ou arquivos de log.
 - **Comandos:** `git init`, `add`, `commit`, `push`.
-

5. Desenvolvimento Frontend I: Estrutura Semântica (HTML5)

5.1 Anatomia e Renderização

- **<!DOCTYPE html>**: Define o modo de renderização moderno. **Atenção:** A ausência desta instrução coloca o navegador em "**Quirks Mode**", quebrando padrões de CSS moderno e causando exibição inconsistente.
- **Tags Globais:** **<html>** (raiz), **<head>** (metadados e vínculos), **<body>** (conteúdo visível).

5.2 Semântica vs. "Divite"

O uso de tags semânticas (**<header>**, **<nav>**, **<main>**, **<section>**, **<article>**, **<footer>**) é superior a **<div>** genéricas para SEO e leitores de tela.

5.3 Tags Essenciais e Atributos Críticos

- **Imagens:** **** deve possuir o atributo **alt**. Se a imagem for meramente decorativa, use **alt=""** (vazio) para que leitores de tela a ignorem.
 - **Formulários:** **<label>** deve possuir o atributo **for** vinculado ao **id** do **<input>**, garantindo que o clique no texto foque o campo.
 - **Listas:** ****, ****, ****.
 - **Tabelas:** **<table>**, **<tr>**, **<td>**.
-

6. Desenvolvimento Frontend II: Estilização Profissional (CSS3)

6.1 Seletores e Especificidade

- **Hierarquia:** ID (#) > Classe (.) > Tag (p).
- **Padrão de Mercado:** Priorize o uso de **Classes** para garantir reusabilidade e evitar a rigidez excessiva do ID.
- **Cascata:** Em regras de mesmo peso, a última declarada prevalece.

6.2 Box Model e Margin Collapse

Todo elemento é uma caixa composta por **Content**, **Padding (interno)**, **Border** e **Margin (externo)**.

- **Pro Tip:** Entenda o **Margin Collapse** (Colapso de Margens). Quando dois elementos verticais se encontram, a margem maior prevalece em vez de somarem-se, o que pode alterar o layout esperado.

6.3 Design System e Variáveis

Use um arquivo **style.css** externo. Defina paletas de cores e fontes no **:root** através de variáveis (ex: **--cor-primaria: #0056b3;**) para facilitar mudanças globais rápidas.

7. Layouts Modernos: Flexbox e Grid

7.1 Flexbox (Unidimensional)

Ideal para alinhar itens em uma direção (linha ou coluna).

- **Propriedades:** **justify-content** (eixo principal), **align-items** (eixo transversal), **flex-direction**.
- **Uso:** Alinhamento de menus, botões internos e micro-componentes.

7.2 CSS Grid (Bidimensional)

Focado no layout macro da página, controlando simultaneamente linhas e colunas.

- **Tracks e Unidades:** Use a unidade fracionária (**fr**) para layouts fluidos.
- **Funções de Especialista:** Utilize **repeat()** para simplificar colunas idênticas e **minmax()** para garantir que colunas não fiquem menores que o conteúdo essencial.

- **Grid Areas:** `grid-template-areas` permite "desenhar" o layout de forma visual no CSS.

7.3 Estratégia Combinada

Use **Grid** para a estrutura macro (posicionamento de header, sidebar, main e footer) e **Flexbox** para micro-alinhamentos dentro dos componentes do Grid.

8. Responsividade, Frameworks e Acessibilidade

8.1 Mobile-First e Media Queries

O desenvolvimento deve começar pela tela menor. Adote os breakpoints: **576px** (mobile landscape), **768px** (tablets), **1024px** (desktop).

8.2 Bootstrap 5

Integre via **CDN** para performance. Utilize o sistema de **12 colunas** e componentes prontos (Navbar, Modais, Cards). O Bootstrap facilita o cumprimento de requisitos de usabilidade em prazos curtos.

8.3 Engenharia de Acessibilidade (WCAG 2.1)

Siga os princípios **POUR** (Perceptível, Operável, Compreensível, Robusto).

- **A Regra de Ouro do ARIA:** "O melhor ARIA é nenhum ARIA". Priorize sempre o HTML semântico nativo antes de recorrer a atributos ARIA.
 - **Contraste:** O alvo mínimo para conformidade Nível AA é uma proporção de **4.5:1**.
 - **Ferramentas:** Use o **Lighthouse** (DevTools) para auditorias automatizadas e o **WebAIM Contrast Checker**.
-

9. Critérios de Qualidade e Entrega de Mercado

9.1 Checklist de Excelência Profissional

- **Clean Code:** Código identado, sem redundâncias e bem comentado.
- **Semântica Validada:** Uso de tags HTML5 em vez de "divite".
- **Contraste e Acessibilidade:** Testes aprovados no Lighthouse.
- **Responsividade:** Zero barras de rolagem horizontal em qualquer resolução.
- **Versionamento Semântico:** Histórico de commits organizado com prefixos claros:
 - `feat:` para novas funcionalidades.
 - `docs:` para documentação.
 - `fix:` para correções de bugs.
- **Segurança:** Implementação de PDO no backend para proteção de dados.